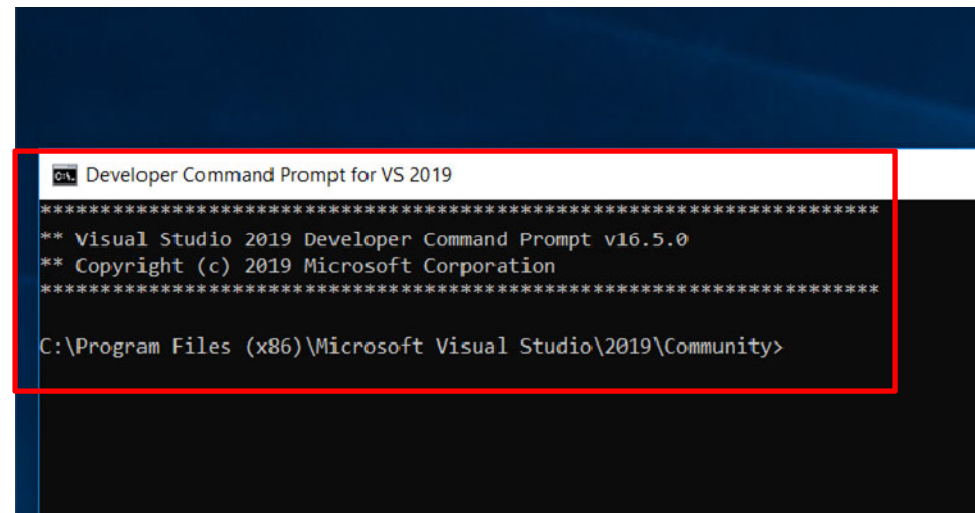
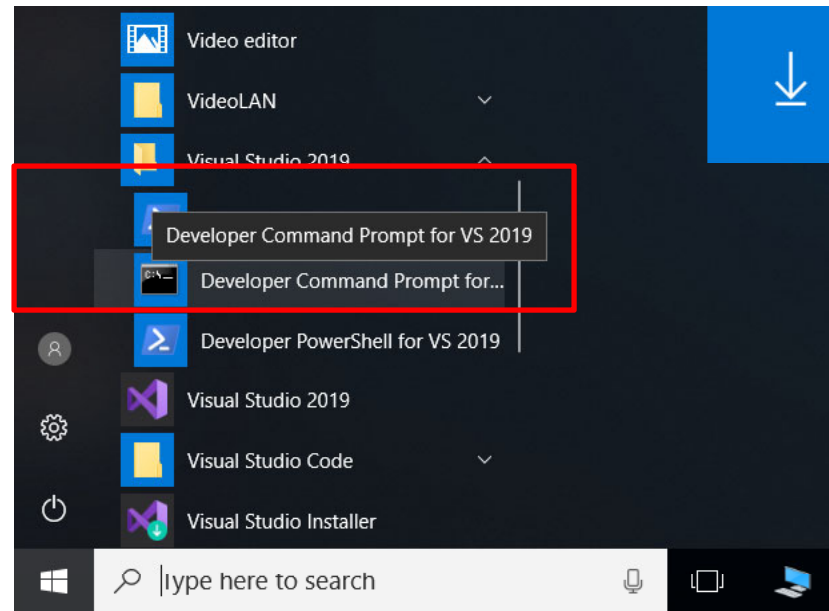
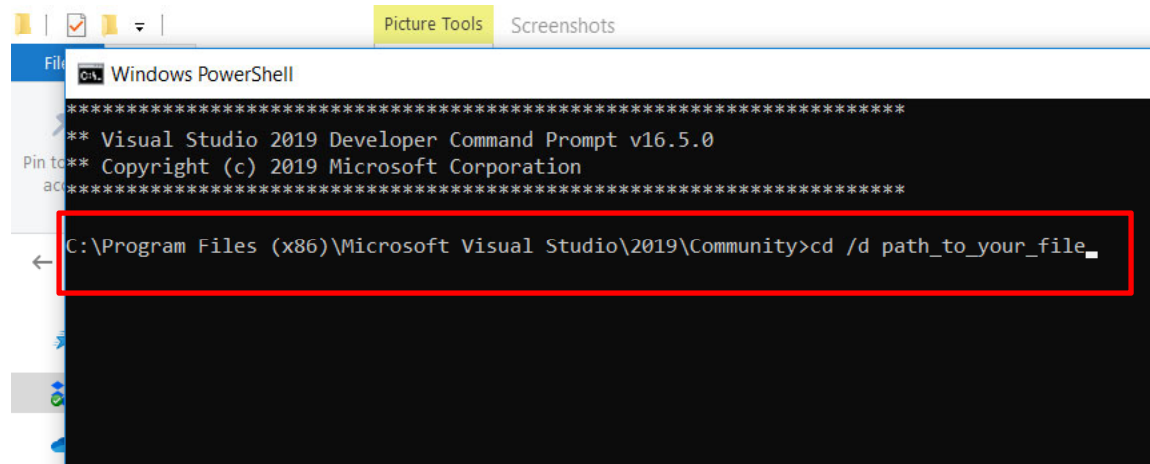


Assembly,
Type and Type members

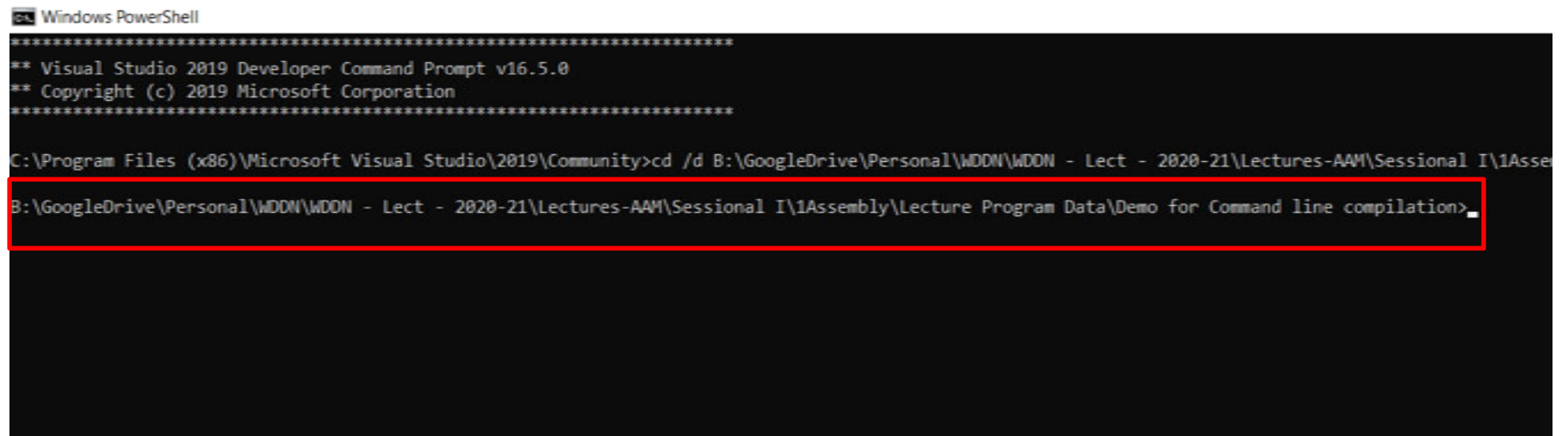
```
1 using System;
2
3 namespace ConsoleApplication1
4 {
5     class Program
6     {
7         static void Main(string[] args)
8         {
9             Console.WriteLine("DDU");
10            Console.ReadLine();
11        }
12    }
13 }
14
```





Windows PowerShell

```
*****  
** Visual Studio 2019 Developer Command Prompt v16.5.0  
** Copyright (c) 2019 Microsoft Corporation  
*****  
C:\Program Files (x86)\Microsoft Visual Studio\2019\Community>cd /d path_to_your_file_
```



Windows PowerShell

```
*****  
** Visual Studio 2019 Developer Command Prompt v16.5.0  
** Copyright (c) 2019 Microsoft Corporation  
*****  
C:\Program Files (x86)\Microsoft Visual Studio\2019\Community>cd /d B:\GoogleDrive\Personal\WDDN\WDDN - Lect - 2020-21\Lectures-AAM\Sessional I\1Asse  
B:\GoogleDrive\Personal\WDDN\WDDN - Lect - 2020-21\Lectures-AAM\Sessional I\1Assembly\Lecture Program Data\Demo for Command line compilation>
```

mpilation

The screenshot shows a Windows File Explorer window and a Windows PowerShell terminal window. The File Explorer window displays a directory listing with the following columns: Name, Date modified, Type, and Size. The files listed are 'Program.cs' (1 KB) and 'Program.exe' (4 KB). The PowerShell window shows the command 'csc Program.cs' being executed, with the output 'Microsoft (R) Visual C# Compiler version 3.5.0-beta4-20153-05 (20b9af91) Copyright (C) Microsoft Corporation. All rights reserved.'

File Explorer Directory Listing:

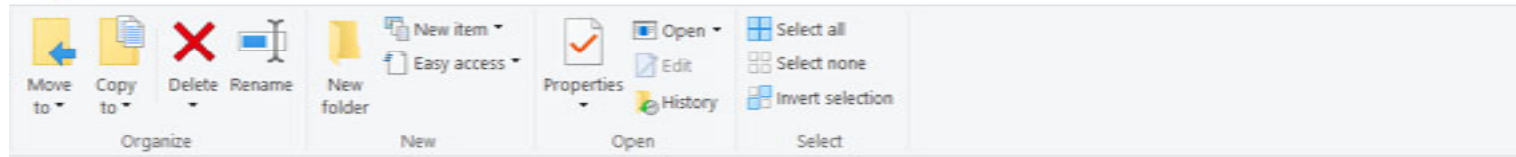
Name	Date modified	Type	Size
Program.cs	13-07-2018 11:02 ...	Visual C# Source F...	1 KB
Program.exe	02-07-2020 03:32 ...	Application	4 KB

Windows PowerShell Command Prompt:

```
*****  
** Visual Studio 2019 Developer Command Prompt v16.5.0  
** Copyright (c) 2019 Microsoft Corporation  
*****  
  
C:\Program Files (x86)\Microsoft Visual Studio\2019\Community>cd /d B:\GoogleDrive\Personal\WDDN\WDDN - Lect - 2020-21\L  
ectures-AAM\Sessional I\1Assembly\Lecture Program Data\Demo for Command line compilation  
  
B:\GoogleDrive\Personal\WDDN\WDDN - Lect - 2020-21\Lectures-AAM\Sessional I\1Assembly\Lecture Program Data\Demo for Com  
and line compilation>csc Program.cs  
Microsoft (R) Visual C# Compiler version 3.5.0-beta4-20153-05 (20b9af91)  
Copyright (C) Microsoft Corporation. All rights reserved.  
  
B:\GoogleDrive\Personal\WDDN\WDDN - Lect - 2020-21\Lectures-AAM\Sessional I\1Assembly\Lecture Program Data\Demo for Comm  
and line compilation>
```

Application Tools Demo for Command line compilation

Manage



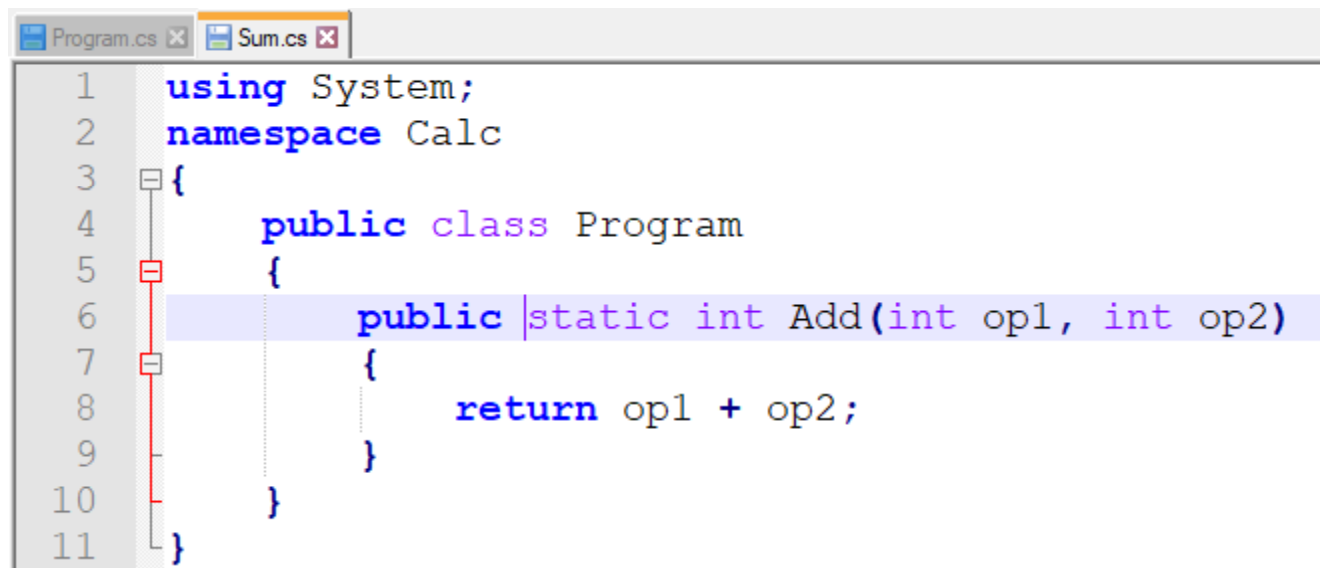
z) > GoogleDrive > Personal > WDDN > WDDN - Lect - 2020-21 > Lectures-AAM > Sessional I > 1Assembly > Lecture Program Data > Demo for Command line

Name	Date modified	Type	Size
Program.cs	13-07-2018 11:02 ...	Visual C# Source F...	1 KB
Program.exe	02-07-2020 03:32 ...	Application	4 KB

B:\GoogleDrive\Personal\WDDN\WDDN - Lect - 2020-21\Lectures-AAM\Sessional I\1Assembly\Lecture Program Data\Demo for Co...

DDU

Ex. 1



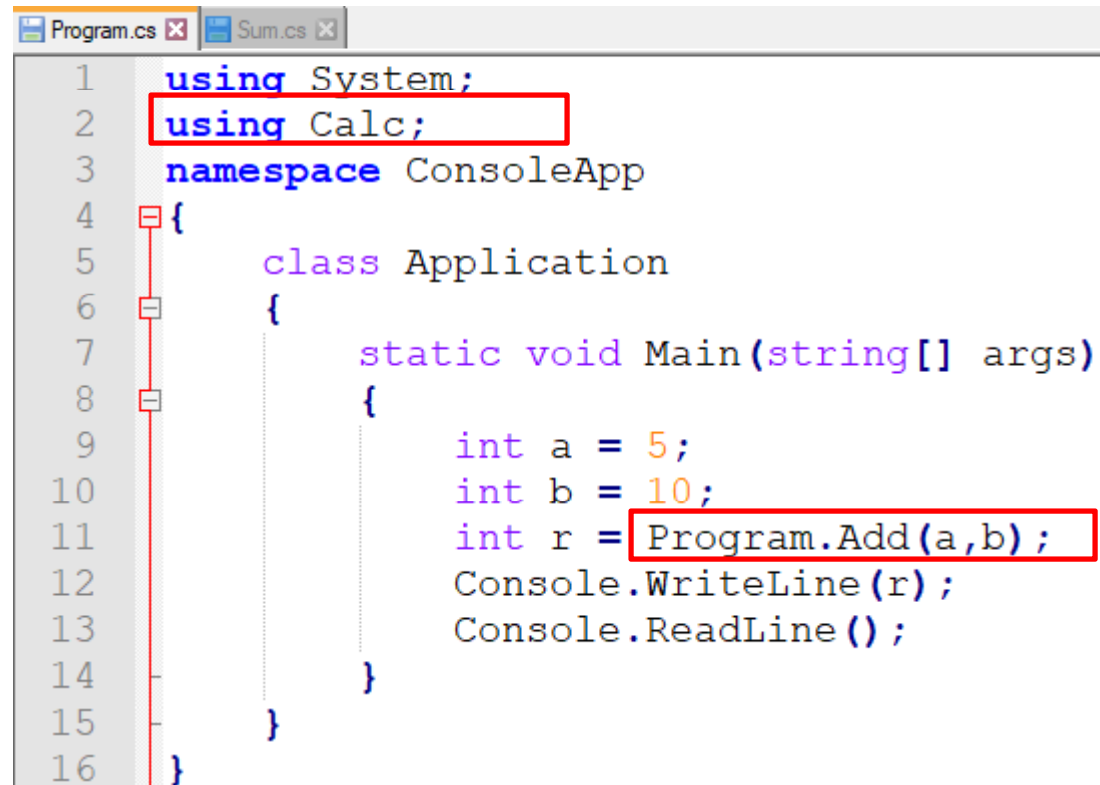
```
1 using System;
2 namespace Calc
3 {
4     public class Program
5     {
6         public static int Add(int op1, int op2)
7         {
8             return op1 + op2;
9         }
10    }
11 }
```

Ex. 1

```
E:\GoogleDrive\Personal\BTech\WDDN\2022-23\Labs\Lab2>csc /t:library Sum.cs  
Microsoft (R) Visual C# Compiler version 3.4.1-beta4-19614-01 (16504609)  
Copyright (C) Microsoft Corporation. All rights reserved.
```

 Program.cs	04-Jul-22 3:07 PM	Visual C# Source F...	1 KB
 Sum.cs	04-Jul-22 3:04 PM	Visual C# Source F...	1 KB
 Sum.dll	04-Jul-22 3:07 PM	Application extens...	3 KB

Ex. 1



```
1  using System;
2  using Calc;
3  namespace ConsoleApp
4  {
5      class Application
6      {
7          static void Main(string[] args)
8          {
9              int a = 5;
10             int b = 10;
11             int r = Program.Add(a,b);
12             Console.WriteLine(r);
13             Console.ReadLine();
14         }
15     }
16 }
```

Ex. 1

```
E:\GoogleDrive\Personal\BTech\WDDN\2022-23\Labs\Lab2>csc Program.cs /r:Sum.dll
Microsoft (R) Visual C# Compiler version 3.4.1-beta4-19614-01 (16504609)
Copyright (C) Microsoft Corporation. All rights reserved.
```

 Program.cs	04-Jul-22 3:07 PM	Visual C# Source F...	1 KB
 Program	04-Jul-22 3:08 PM	Application	4 KB
 Sum.cs	04-Jul-22 3:04 PM	Visual C# Source F...	1 KB
 Sum.dll	04-Jul-22 3:07 PM	Application extens...	3 KB

```
E:\GoogleDrive\Personal\BTech\WDDN\2022-23\Labs\Lab2>Program.exe
15
```

Ex. 2

- Create assembly `calc.dll` which contains functions to add, subtract, multiply and divide two numbers.
- Create `program.cs` which contains main method and uses `calc.dll` file.

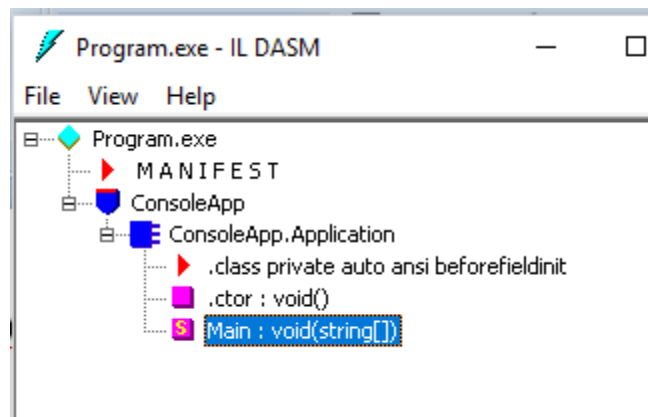
Ex. 3

- Create assembly `sum.dll` and `sub.dll` which contains functions to add two numbers and subtract two numbers respectively.
- Create `program.cs` which contains main method and uses `sum.dll` and `sub.dll` files.

ILDASM

```
E:\GoogleDrive\Personal\BTech\WDDN\2022-23\Labs\Lab2\ex3>ildasm Program.exe
E:\GoogleDrive\Personal\BTech\WDDN\2022-23\Labs\Lab2\ex3>Program.exe
15
-5
```

ILDASM



```
ConsoleApp.Application::Main : void(string[])
Find Find Next

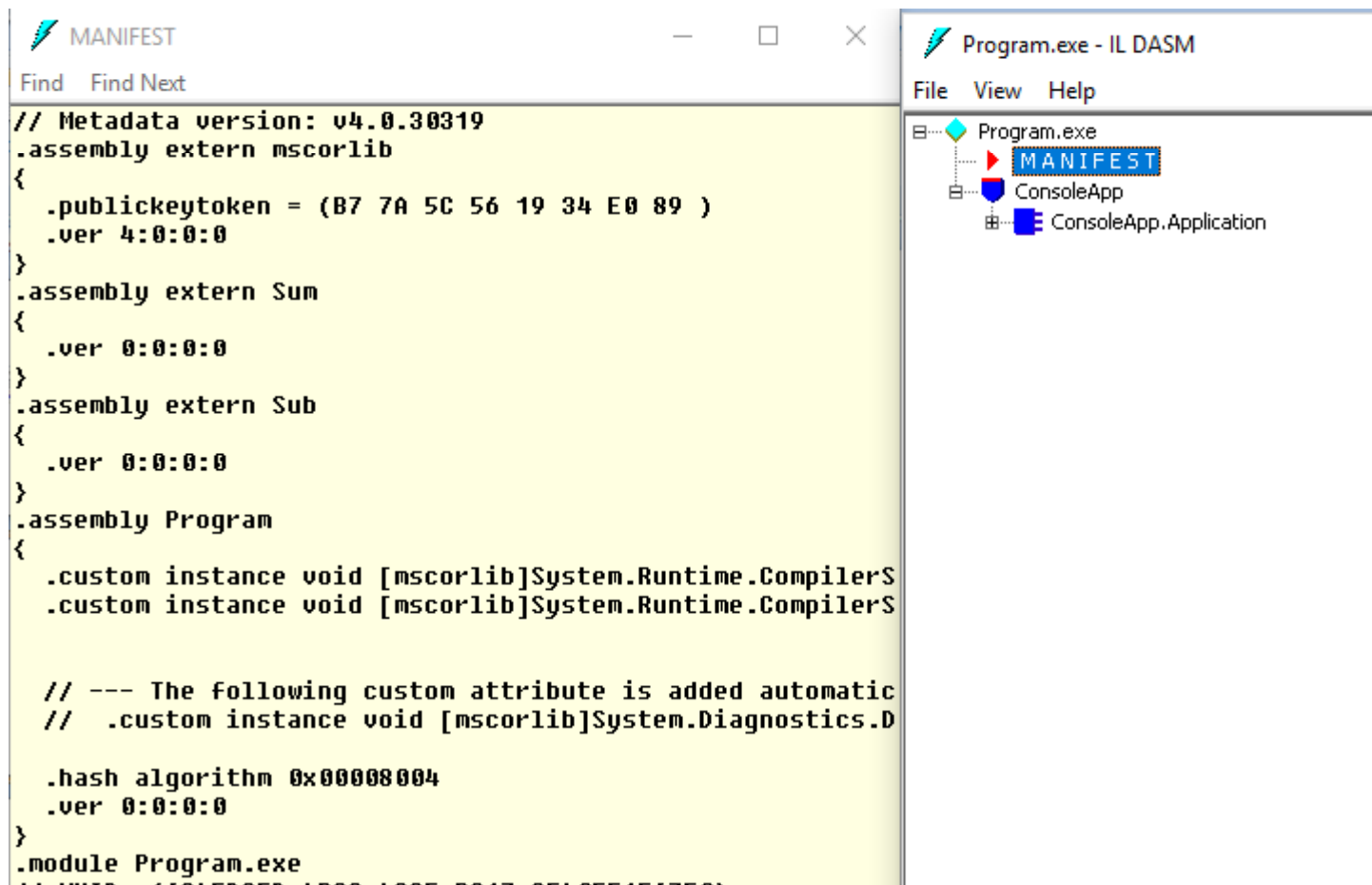
.method private hidebysig static void Main(string[] args) cil managed
{
    .entrypoint
    // Code size          43 (0x2b)
    .maxstack 2
    .locals init (int32 U_0,
                  int32 U_1,
                  int32 U_2,
                  int32 U_3)

    IL_0000: nop
    IL_0001: ldc.i4.5
    IL_0002: stloc.0
    IL_0003: ldc.i4.s 10
    IL_0005: stloc.1
    IL_0006: ldloc.0
    IL_0007: ldloc.1
    IL_0008: call      int32 [Sum]Sum.Program1::Add(int32,
                                                int32)

    IL_000d: stloc.2
    IL_000e: ldloc.0
    IL_000f: ldloc.1
    IL_0010: call      int32 [Sub]Sub.Program2::Sub(int32,
                                                int32)

    IL_0015: stloc.3
    IL_0016: ldloc.2
    IL_0017: call      void [mscorlib]System.Console::WriteLine(int32)
    IL_001c: nop
    IL_001d: ldloc.3
    IL_001e: call      void [mscorlib]System.Console::WriteLine(int32)
    IL_0023: nop
    IL_0024: call      string [mscorlib]System.Console::ReadLine()
    IL_0029: pop
    IL_002a: ret
} // end of method Application::Main
```

ILDASM



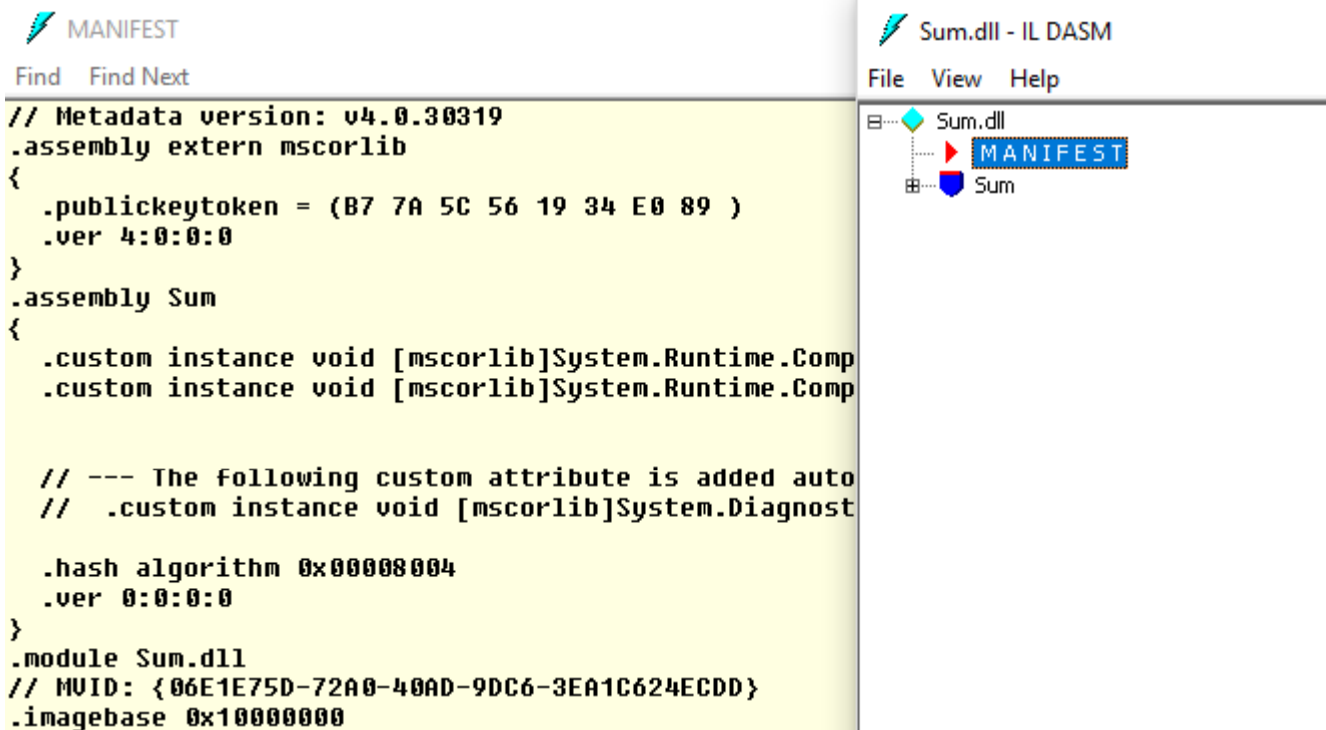
?

- Say, your friend Mr. Anders Hejlsberg has created an assembly (.dll) file implementing robust and time efficient sorting algorithm for floating point numbers.
- He has shared the assembly with you.
- Can you use (i.e. access sorting method) it in your program? If no, why? If yes, how?

ILDASM

```
E:\GoogleDrive\Personal\BTech\WDDN\2022-23\Labs\Lab2\ex3>ildasm Sum.dll
```

ILDASM



ILDASM

The screenshot displays the ILDASM application window, which is divided into two main panes. The left pane shows the disassembled code for the method `Sum.Program1::Add(int32, int32)`. The right pane shows the assembly tree for `Sum.dll`.

Left Pane: Disassembly

Method: `Sum.Program1::Add(int32, int32)`

Find Find Next

```
.method public hidebysig static int32 Add(int32 op1,
                                           int32 op2) cil managed
{
    // Code size          9 (0x9)
    .maxstack 2
    .locals init (int32 V_0)
    IL_0000: nop
    IL_0001: ldarg.0
    IL_0002: ldarg.1
    IL_0003: add
    IL_0004: stloc.0
    IL_0005: br.s      IL_0007
    IL_0007: ldloc.0
    IL_0008: ret
} // end of method Program1::Add
```

Right Pane: Assembly Tree

Sum.dll - IL DASM

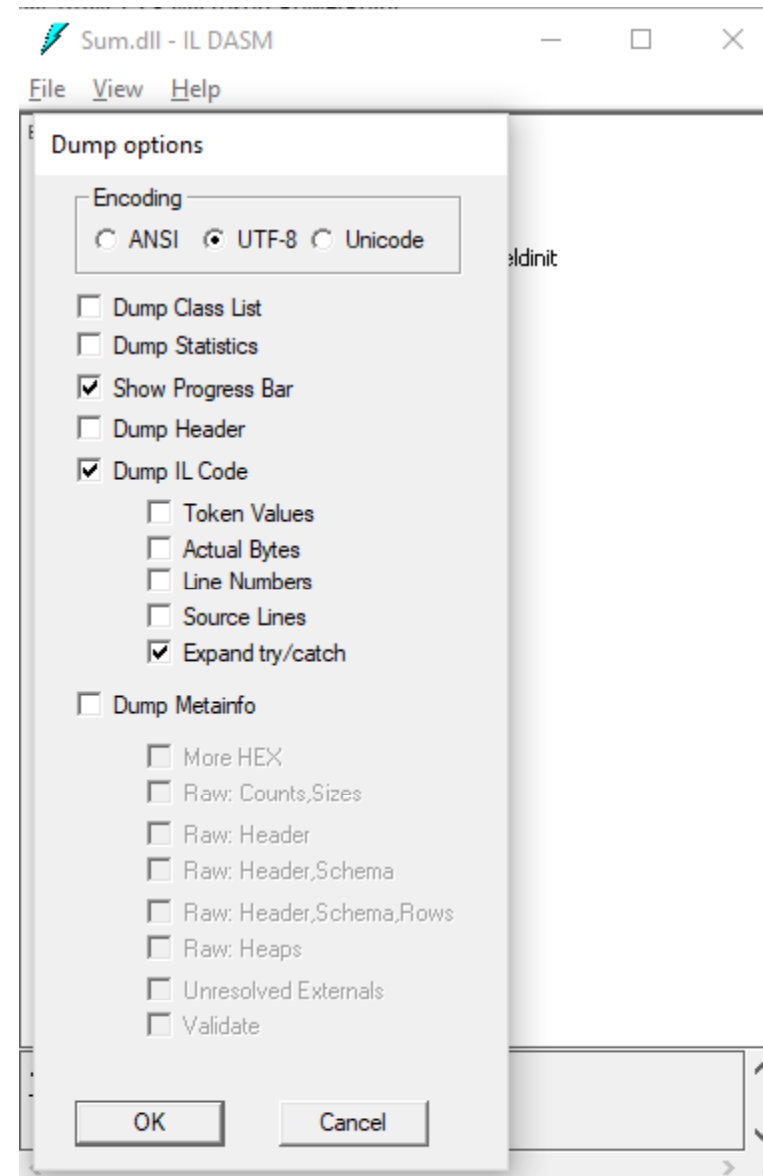
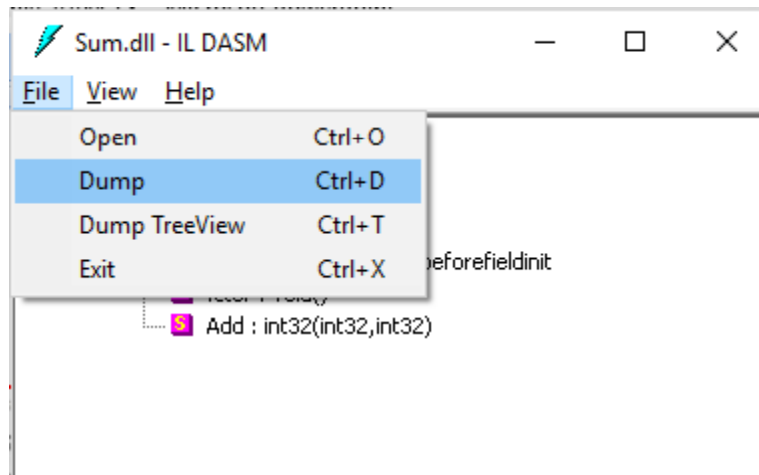
File View Help

- Sum.dll
 - MANIFEST
 - Sum
 - Sum.Program1
 - .class public auto ansi beforefieldinit
 - .ctor : void()
 - Add : int32(int32, int32)

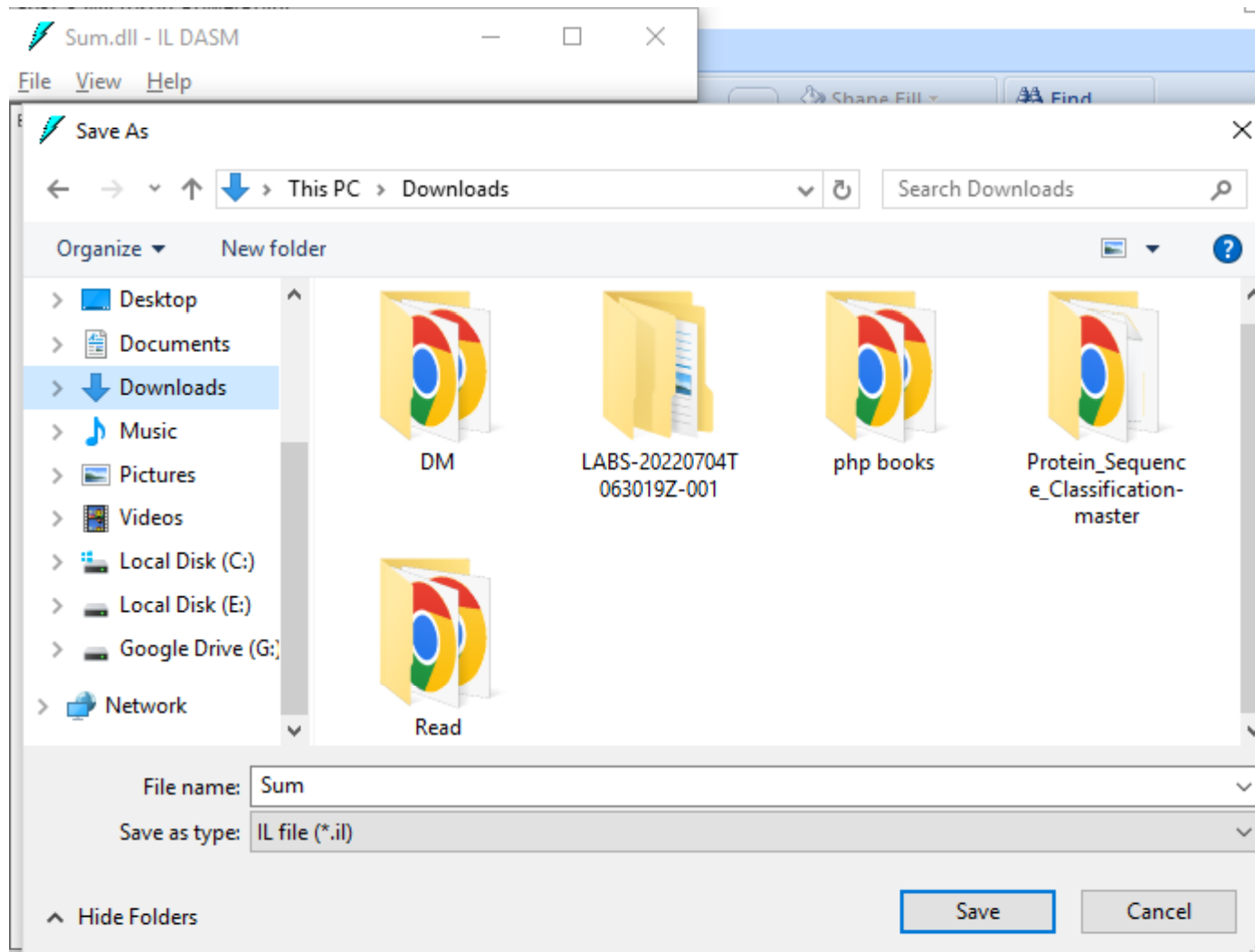
ILDASM.EXE

- We use ILDASM (intermediate language disassembler) to peek at the assembly manifest and IL.
- You can also use this tool to export manifest and IL to a text file
- MSIL + namespace + types + ...

ILDASM



ILDASM



ILDASM



Sum.il

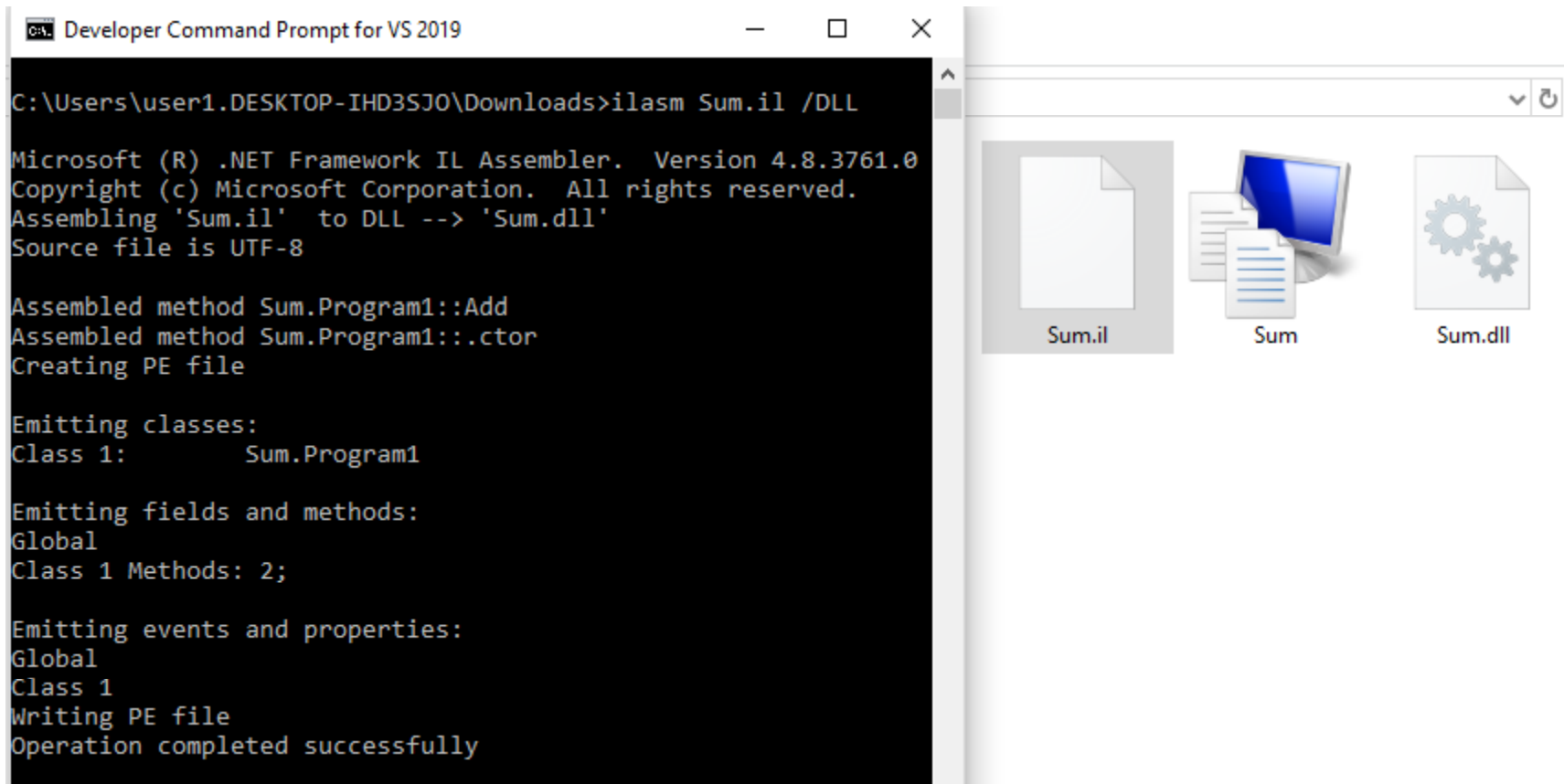


Sum

ILASM.EXE

- We use ILASM.EXE to reconstruct an assembly from a text file that contains manifest and IL

ILASM.EXE



The image shows a Windows Developer Command Prompt window titled "Developer Command Prompt for VS 2019" with a black background and white text. The command prompt shows the execution of the command `ilasm Sum.il /DLL` in the directory `C:\Users\user1.DESKTOP-IHD3SJ0\Downloads`. The output displays the version of the .NET Framework IL Assembler (4.8.3761.0) and the process of assembling `Sum.il` into a DLL named `Sum.dll`. It lists the assembly of methods `Sum.Program1::Add` and `Sum.Program1::.ctor`, the creation of the PE file, and the emission of classes, fields, methods, and events. The process concludes with "Operation completed successfully".

```
C:\Users\user1.DESKTOP-IHD3SJ0\Downloads>ilasm Sum.il /DLL

Microsoft (R) .NET Framework IL Assembler. Version 4.8.3761.0
Copyright (c) Microsoft Corporation. All rights reserved.
Assembling 'Sum.il' to DLL --> 'Sum.dll'
Source file is UTF-8

Assembled method Sum.Program1::Add
Assembled method Sum.Program1::.ctor
Creating PE file

Emitting classes:
Class 1:      Sum.Program1

Emitting fields and methods:
Global
Class 1 Methods: 2;

Emitting events and properties:
Global
Class 1
Writing PE file
Operation completed successfully
```

To the right of the command prompt, a file explorer window shows the same directory. It contains three files: `Sum.il` (represented by a document icon), `Sum` (represented by a document and a monitor icon), and `Sum.dll` (represented by a document and gear icon).

Type and Type member

- Customer is the Type and fields, properties and methods are Type members

```
public class Customer
{
    #region fields
    private int _id;
    private string _fName;
    private string _lName;
    #endregion

    #region properties
    public int id { get; set; }
    public string fName { get; set; }
    public string lName { get; set; }
    #endregion

    #region methods
    public string GetFullName()
    {
        return this._fName + " " + this._lName;
    }
    #endregion
}
```

Type and Type member

- In general classes, structs, enums, interfaces, delegates are called as types and fields, properties, constructors, methods etc..., that normally reside in a type are called as type members
- In C# there are 5 different access modifiers
 - Private
 - Protected
 - Internal
 - Protected Internal
 - Public
- Type members can have all the access modifiers, where as types can have only 2 (internal, public) of the 5 access modifiers

Access specifiers/ modifiers

- Promote encapsulation
 - Type or type member may limit the accessibility to other types and assemblies
- There are 5 different access modifiers in C#
 - Public
 - Private
 - Protected
 - Internal
 - Protected Internal

public

- Access not limited [Fully Accessible]

```
class class1 {}
```

```
public class class2 {}
```

```
//class2 is accessible from outside its  
//assembly, not class 1
```

private

- Accessible only within containing type
- Default for member of a class or struct

```
class classA { int x; }
```

```
//classA does not expose field x to other types  
//in the same assembly
```

protected

- Accessible only within containing type or subclass

```
class BaseClass
```

```
{
```

```
    void Foo()    {} //Foo is private (default)
```

```
    protected void Bar() {}
```

```
}
```

```
class SubClass : BaseClass
```

```
{
```

```
    void Test1() { Foo(); } //error – cannot access Foo
```

```
    void Test2() { Bar(); } // OK
```

```
}
```

internal

- Accessible only within containing assembly

```
//Assembly1.CS  
//Compile with: /t:library
```

```
internal class BaseClass  
{  
    public int intM=0;  
}
```

```
//Assembly1_a.CS  
//Compile with: /r:Assembly1.dll
```

```
class TestAccess  
{  
    static void Main()  
    {  
        BaseClass myBase=new BaseClass();  
    }  
}
```


Cont...

```
//Assembly2.CS
//Compile with: /t:library

public class BaseClass
{
    internal static int intM=0;
}

//Assembly2_a.CS
//Compile with: /r:Assembly2.dll

class TestAccess
{
    static void Main()
    {
        BaseClass myBase=new BaseClass();
        BaseClass.intM=444;
    }
}
```

protected internal

- The union of protected and internal accessibility.

```

//Assembly1.CS
//Compile with: /t:library

public class BaseClass
{
    protected internal int myValue=0;
}
class TestAccess
{
    void Access()
    {
        BaseClass baseObject = new BaseClass();
        baseObject.myBase = 5;
    }
}

//Assembly2.CS
//Compile with: /r:Assembly1.dll

class DerivedClass : BaseClass
{
    static void Main()
    {
        BaseClass baseObject=new BaseClass();
        DerivedClass derivedObject = new DerivedClass();
        baseObject.myValue=10;
        derivedObject.myValue=10;
    }
}

```

```
Assembly1.cs x Assembly2.cs x Assembly3.cs x
1 public class BaseClass
2 {
3     protected internal int MyValue=0;
4 }
5
6 class TestAccess
7 {
8     void Access ()
9     {
10         BaseClass bObj=new BaseClass ();
11         bObj.MyValue = 5;
12     }
13 }
```

```
Assembly1.cs x Assembly2.cs x Assembly3.cs x new1 x
1 public class DerivedClass: BaseClass
2 {
3     static void Main()
4     {
5         // //Case 1
6         // BaseClass baseObj=new BaseClass();
7         // DerivedClass derivedObj=new DerivedClass();
8         // baseObj.MyValue = 10;
9         // derivedObj.MyValue = 10;
10        // //Case 2
11        // BaseClass baseObj=new DerivedClass();
12        // baseObj.MyValue = 10;
13        //Case 3
14        DerivedClass derivedObj=(DerivedClass)new BaseClass();
15        derivedObj.MyValue = 10;
16    }
17 }
```

```
Assembly1.cs Assembly2.cs Assembly3.cs new 1
1 class DDerivedClass: DerivedClass
2 {
3     static void Main()
4     {
5         DerivedClass derivedObj = new DerivedClass();
6         DDerivedClass dderivedObj = new DDerivedClass();
7
8         derivedObj.MyValue=20;
9         dderivedObj.MyValue=30;
10    }
11 }
```

Access modifiers for types and types members

- Types: Class, Struct, etc...
 - Only public and internal access modifier is allowed
 - Internal is default for type
- Private is default for type members